```
      t.citext :email
    end
  end
end
```

Or create the table with SQL directly by running `CREATE TABLE temp.customers (email citext);`.

Once you've added the `temp.customers` table, try inserting some data into it. Run the following Insert statement from psql.

```
INSERT INTO temp.customers (email) VALUES ('Customer1@example.com');
```

Now try querying for the row using a lower case "c" in the WHERE clause like `WHERE email = 'customer1@example.com'`.

```
SELECT * FROM temp.customers WHERE email = 'customer1@example.com';
```

You should have the row you inserted as a query result. Without `citext` there would have been no results.

For `citext` columns, Unique constraints enforce unique values regardless of their casing. Add a Unique constraint and try to insert a customer with an email address with the same text but different casing, and it will be prevented.

While `citext` is a great solution for retaining original casing and still enforcing Uniqueness, there may be other use cases where you wish to perform a normalization process on data prior to inserting it.

How can that be achieved?

Storing Transformations In Generated Columns

PostgreSQL 12 added a feature called *Generated Columns* and it's even supported by Ruby on Rails. The Rails support for Generated Columns[13] is called `virtual` columns. These columns can be created using Active Record Migrations.

In this section you'll create a generated column to see how to use this feature.

Call your Generated Column `email_downcased`. If you've just added `temp.customers` in a Rails migration to Rideshare, roll it back by running `bin/rails db:rollback` in your terminal or drop the table with SQL.

Now you can replace the `create_table` portion from before adding the `email` column with the `string` type and also the `email_downcased` column.

13. https://guides.rubyonrails.org/active_record_postgresql.html#generated-columns